Jhanvi Cars

Complete Project Overview — Frontend, Backend, Kubernetes, Networking, CI/CD and AWS

Document purpose

This document consolidates the current Jhanvi Cars project architecture, implementation status, development environment, deployment approach and planned production architecture. Sensitive credentials and API secrets are intentionally excluded.

1. Project Overview

Architecture: Jhanvi Cars is a ride-booking application with a React Native mobile frontend, Node.js backend, PostgreSQL database, Google Maps/Places/Routes integrations, Kubernetes deployment, Docker images, GitHub Actions CI/CD, and an AWS production target.

Core user flow: A customer authenticates, selects a pickup location, selects a destination, receives route/fare information, selects a vehicle, and continues through booking.

Brand: The application and infrastructure are being developed under the Jhanvi Cars name.

2. React Native Frontend

Technology: React Native 0.74.0 with React 18.2.0.

Project path: D:\jhanvi-dev\jhanvi-cars on the Windows development machine.

Structure: The project uses src/components, navigation, services, context, hooks, utils, constants, assets, and customer/driver/auth screen areas.

API configuration: The development frontend currently uses the Windows-reachable backend endpoint http://192.168.1.5:30080.

Release behavior: A release APK does not need Metro or an ADB reverse tunnel. The phone must be able to reach the configured backend endpoint.

3. Booking Screen

Pickup: Pickup is represented separately from current GPS location through pickupLocation. The user can use GPS, search for a pickup place, or use map selection.

GPS: GPS selection sets currentLocation and pickupLocation. The current implementation can display coordinates; reverse geocoding is planned to provide a readable place name.

Destination: Destination search uses the existing PlacesModule and stores destinationLocation.

Fare calculation: Driving-route calculation is triggered using pickupLocation and destinationLocation together with the selected vehicle.

Continue: The booking flow validates pickupLocation and passes it forward instead of requiring currentLocation.

Map selection: The map-selection UI uses a center pin and controls outside MapView. This avoids the React Native Maps child-parent crash caused by placing ordinary Views inside MapView.

4. Google Places Native Module

File: android/app/src/main/java/com/bangalorecabbookingapp/PlacesModule.kt

Search: The module provides searchPlaces using Google Places Autocomplete, biased around the supplied coordinates and restricted to India.

Details: getPlaceDetails fetches place ID, display name, formatted address, latitude and longitude.

Planned enhancement: Add reverseGeocode(latitude, longitude) so GPS-selected coordinates can be converted into a human-readable address. This native change has not yet been confirmed as implemented.

5. Maps and Routing

Google services: The project uses Google Maps SDK for Android, Places API (New), and Routes API.

Routing: The backend has a Google Routes integration through services/googleRoutes.js and uses the GOOGLE_ROUTES_API_KEY environment variable.

Development result: The application has been able to register and book a cab, and Google Routes is currently functioning in the development environment.

6. Vehicle and Fare Model

Vehicle selection: The booking screen includes vehicle selection and uses the selected vehicle as an input to fare calculation.

Production direction: A production ride-hailing implementation should keep pricing logic authoritative on the backend. The frontend should display the calculated quote returned by the backend rather than being the final source of truth.

Typical production inputs: Potential pricing inputs include base fare, distance, duration, vehicle category, minimum fare, surge/dynamic pricing, tolls, taxes, and applicable platform fees. Exact business rules should be defined by the application owner.

7. Authentication

Frontend: Authentication is provided through AuthContext and AsyncStorage. The token key is @jhanvi_car_token.

API: Login, registration and current-user requests are handled through the backend authentication endpoints.

Registration contract: The backend registration contract expects the mobile field rather than phone.

Security: JWT signing uses a Kubernetes Secret-backed JWT_SECRET. Actual secret values are intentionally omitted from this document.

8. Backend and PostgreSQL

Backend: The recovered Node.js backend contains database configuration, controllers, routes, middleware, server.js and Google Routes integration.

Database: PostgreSQL 16 is used with the cabbooking database.

Configuration: The backend receives DB_HOST, DB_PORT, DB_NAME, DB_USER and DB_PASSWORD through environment configuration.

Kubernetes: PostgreSQL runs as a deployment with a persistent volume claim, while the backend connects through the Kubernetes postgres service.

Data: The project has a PostgreSQL backup containing application tables such as bookings, users and vehicles.

9. Kubernetes / Minikube

Namespace: jhanvi-cars.

Backend: Deployment jhanvi-cars-backend, exposed through a NodePort service.

Database: Deployment postgres with service postgres and PVC postgres-pvc.

Health: The backend exposes /health and has been tested successfully.

Current NodePort: The backend is currently exposed internally through NodePort 31824 in Minikube; Windows reaches it through the forwarding chain described below.

10. Development Networking

Windows: Windows Wi-Fi address used by the phone: 192.168.1.5.

Ubuntu VM: Host-only address: 192.168.56.4.

Windows host-only: Windows host-only address: 192.168.56.3.

Minikube: Minikube address: 192.168.49.2.

Flow: Phone/Windows → Windows 192.168.1.5:30080 → Ubuntu VM 192.168.56.4:30080 → Minikube 192.168.49.2:31824 → backend → PostgreSQL.

Forwarding: IPv4 forwarding and persistent iptables rules are configured. A systemd network service dynamically configures forwarding based on the Minikube IP and Docker bridge.

11. Automatic Minikube Startup

Service: minikube-jhanvi-cars.service starts the Minikube profile using the Docker driver after Docker starts.

Network service: jhanvi-cars-network.service configures the required forwarding after Minikube becomes available.

Operational note: After a VM reboot, Minikube can take several minutes to initialize before the network-forwarding service can complete.

12. Docker

Images: Jhanvi Cars backend images have been built locally and published to Docker Hub.

Repository: The backend Docker image is maintained under the jhanviadmin Docker Hub namespace.

Optimization: Production images should use appropriate base images, multi-stage builds where useful, minimal runtime dependencies, deterministic versions and Docker layer caching.

13. GitHub Actions CI/CD

Repository: The project is maintained in the jttadmin/jhanvi-cars GitHub repository.

Build pipeline: GitHub Actions builds and pushes the Docker image using Docker Hub credentials stored as GitHub secrets.

Runner: A self-hosted GitHub Actions runner has been configured on the Ubuntu environment.

Deployment direction: The development pipeline can evolve into an AWS deployment pipeline that builds, scans, publishes and deploys versioned images.

14. Terraform / AWS

Terraform: Terraform v1.16.2 with AWS provider v6.64.0 has been used for infrastructure work.

Region: The development AWS infrastructure has used ap-south-2.

VPC: A VPC using 10.0.0.0/16 was created with public and private subnets across multiple availability zones.

Resources: The Terraform state includes route tables, internet gateway, subnet associations and security groups for ALB, EKS and RDS-related infrastructure.

Cost awareness: NAT gateways, EKS control-plane charges, worker nodes, load balancers, storage and data transfer need to be considered when estimating production cost.

15. Intended Production Architecture

Frontend: React Native Android/iOS application distributed as a release build.

Edge: DNS and HTTPS terminate at a production load-balancing layer.

Application: Containerized backend deployed on Kubernetes/EKS with multiple replicas.

Database: Managed PostgreSQL such as Amazon RDS, using private subnets and appropriate backups.

Networking: Public subnets for internet-facing components and private subnets for application/database components.

Scaling: Horizontal backend scaling, health checks, readiness/liveness probes, autoscaling and controlled rolling deployments.

Observability: Centralized logs, metrics, tracing and alerts should be added before production.

Secrets: Application secrets should be managed through a dedicated secrets mechanism rather than committed to Git.

16. Security

Secrets: Do not commit JWT secrets, database passwords, Google API keys or payment-provider secrets to source control.

API security: Use HTTPS, authentication/authorization, rate limiting, input validation, secure headers and controlled CORS.

Database: Keep PostgreSQL private and restrict security-group/network access to the application layer.

Mobile: Avoid embedding server-side secrets in the APK. Mobile-restricted Google API keys should use appropriate package/SHA-1 restrictions and API restrictions.

17. Current Status

Working: Backend health endpoint, PostgreSQL connectivity, authentication, registration/booking flow, Google Routes integration, Kubernetes backend deployment, Minikube startup automation, Docker publishing and phone-to-backend networking have been established.

In progress: Production-quality pickup selection UX is being refined. GPS pickup currently needs reverse geocoding for a readable place name.

Map search: Map pickup selection is implemented at the UI level but further refinement/testing is planned.

Production: The AWS/EKS architecture is being developed as the production target rather than the current Minikube environment.

18. Roadmap

• Complete GPS reverse geocoding and display a readable pickup address.

• Refine map-based pickup selection and marker behavior.

• Complete production-grade fare/pricing logic on the backend.

• Add robust booking state handling, cancellation and driver-side workflows.

- Add secure payment integration and server-side payment verification if required.
- Containerize and version all production components.
- Deploy the backend to EKS and PostgreSQL to RDS or another managed database.
- Add ALB/Ingress, TLS, DNS, observability, backups and disaster-recovery procedures.
- Build a release APK/AAB and validate the complete mobile-to-production flow.

Security note: This document intentionally excludes passwords, JWT secret values, API keys and other credentials.